CIS 1904: Haskell

Lazy Evaluation

Logistics

- HW04 due yesterday
- HW05 available this evening
- If you’re resubmitting HW02, that’s due March 2
- Next week: Optional review class

What is Lazy Evaluation?

```
lastOrDefault :: List a -> a -> a
lastOrDefault [] def = def
lastOrDefault [h] _ = h
lastOrDefault (h : tl) _ = lastOrDefault tl

addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs
```

What is Lazy Evaluation?

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

addLastOrDefault 2 0 (sort [4,1,3])
```

In *strict* evaluation, arguments must be evaluated before the function can run.

The Alternative: Strict Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

addLastOrDefault 2 0 (sort [4,1,3])
```

The Alternative: Strict Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

addLastOrDefault 2 0 [1,3,4]
```

The Alternative: Strict Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

2 + lastOrDefault 0 ([1,3,4])
```

The Alternative: Strict Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

2 + 4
```

The Alternative: Strict Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

6
```

The Alternative: Strict Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs
```

```
6
```

This is what you see in most programming languages (OCaml, Java, etc.)!

Lazy Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

addLastOrDefault 2 0 (sort [4,1,3])
```

Main idea: a function argument does not get evaluated until it is needed

Lazy Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

addLastOrDefault 2 0 (sort [4,1,3])
```

Main idea: a function argument does not get evaluated until it is needed

Lazy Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

2 + lastOrDefault 0 (sort [4,1,3])
```

Main idea: a function argument does not get evaluated until it is needed

Lazy Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

2 + lastOrDefault 0 (sort [4,1,3])
```

Main idea: a function argument does not get evaluated until it is needed

Lazy Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

2 + lastOrDefault 0 [1,3,4]
```

Main idea: a function argument does not get evaluated until it is needed

Lazy Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

2 + 4
```

Main idea: a function argument does not get evaluated until it is needed

Lazy Evaluation

```
addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs
```

```
6
```

Main idea: a function argument does not get evaluated until it is needed

What is Lazy Evaluation?

Main idea: Haskell does not evaluate function arguments until needed

- In Haskell, arguments are implicitly boxed up inside *thunks*

What is Lazy Evaluation?

Main idea: Haskell does not evaluate function arguments until needed

- In Haskell, arguments are implicitly boxed up inside *thunks*

```
def foo x y = …
```

```
arg1 = f v1
arg2 = g v2
foo arg1 arg2
```

What is Lazy Evaluation?

Main idea: Haskell does not evaluate function arguments until needed

- In Haskell, arguments are implicitly boxed up inside *thunks*
- Unpackaging and evaluating an argument to inspect it is called *forcing* it

```
def foo x y = case x of …
```

```
arg1 = f v1
arg2 = g v2
foo arg1 arg2 → case arg1 of …
```

What is Lazy Evaluation?

Main idea: Haskell does not evaluate function arguments until needed

- In Haskell, arguments are implicitly boxed up inside *thunks*
- Unpackaging and evaluating an argument to inspect it is called *forcing* it

```
def foo x y = case x of …
```

```
arg1 = f v1
arg2 = g v2
foo arg1 arg2 → case arg1 of …
```

What is Lazy Evaluation?

Main idea: Haskell does not evaluate function arguments until needed

- In Haskell, arguments are implicitly boxed up inside *thunks*
- Unpackaging and evaluating an argument to inspect it is called *forcing* it

```
def foo x y = case x of …
```

```
arg1 = f v1
arg2 = g v2
foo arg1 arg2 → case f v1 of …
```

What is Lazy Evaluation?

Main idea: Haskell does not evaluate function arguments until needed

- In Haskell, arguments are implicitly boxed up inside *thunks*
- Unpackaging and evaluating an argument to inspect it is called *forcing* it

```
def foo x y = case x of …
```

```
arg1 = f v1
arg2 = g v2
foo arg1 arg2 → case r1 of …
```

Q: When do lazy expressions get evaluated?

Q: When do lazy expressions get evaluated?

A: When we pattern match on them.

What is Lazy Evaluation?

Main idea: a function call does not get evaluated until its output is needed

```
lastOrDefault :: List a -> a -> a
lastOrDefault [] def = def
lastOrDefault [h] _ = h
lastOrDefault (h : tl) _ = lastOrDefault tl

addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

addLastOrDefault 2 0 (sort [4,1,3])
```

What is Lazy Evaluation?

Main idea: a function call does not get evaluated until its output is needed

```
lastOrDefault :: List a -> a -> a
lastOrDefault [] def = def
lastOrDefault [h] _ = h
lastOrDefault (h : tl) _ = lastOrDefault tl

addLastOrDefault :: Int -> Int -> List Int -> Int
addLastOrDefault i def xs = i + lastOrDefault def xs

addLastOrDefault 2 0 (sort [4,1,3])
```

We need to evaluate to know which case of `lastOrDefault` we're in!

When do lazy expressions get evaluated?

```
foo1 :: Maybe a -> [Maybe a]
foo1 m = [m, m]
```

Will foo1's argument be evaluated during its execution?

When do lazy expressions get evaluated?

```
foo :: Maybe a -> [Maybe a]
foo m = [m, m]
```

Will foo1's argument be evaluated during its execution?

No, we do not need to pattern match on `m` to place it in a list.

When do lazy expressions get evaluated?

```
foo2 :: Maybe a -> [a]
foo2 Nothing  = []
foo2 (Just x) = [x]
```

Will foo2's argument be evaluated during its execution?

When do lazy expressions get evaluated?

```
foo2 :: Maybe a -> [a]
foo2 Nothing  = []
foo2 (Just x) = [x]
```

Will foo2's argument be evaluated during its execution?

Partially – we need to know if its argument is `Nothing` or `Just`, but we do not need to evaluate `x`.

When do lazy expressions get evaluated?

```
foo2 :: Maybe a -> [a]
foo2 Nothing  = []
foo2 (Just x) = [x]


foo3 :: Int -> [Int]
foo3 x = foo2 (Just x^210)
```

Will foo3's argument be evaluated?

When do lazy expressions get evaluated?

```
foo2 :: Maybe a -> [a]
foo2 Nothing  = []
foo2 (Just x) = [x]


foo3 :: Int -> [Int]
foo3 x = foo2 (Just x^210)
```

Will foo3's argument be evaluated?

No – we only need to pattern match to know if we are in the `Nothing` case or the `Just` case. We do not need to match on the integer.

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → addTwo 5 (29 * 29 * … * 29)
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → addTwo 5 (29 * 29 * … * 29)
    → addTwo 5 (841 * … * 29)
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → addTwo 5 (29 * 29 * … * 29)
    → addTwo 5 (841 * … * 29)
    → addTwo 5 (24389 * … * 29)
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → addTwo 5 (29 * 29 * … * 29)
    → addTwo 5 (841 * … * 29)
    → addTwo 5 (24389 * … * 29)
    …
    → addTwo 5 (12693446555…)
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → addTwo 5 (29 * 29 * … * 29)
    → addTwo 5 (841 * … * 29)
    → addTwo 5 (24389 * … * 29)
    …
    → addTwo 5 (12693446555…)
    → 5 + 2
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → addTwo 5 (29 * 29 * … * 29)
    → addTwo 5 (841 * … * 29)
    → addTwo 5 (24389 * … * 29)
    …
    → addTwo 5 (12693446555…)
    → 5 + 2
    → 7
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → addTwo 5 (29 * 29 * … * 29)
    → addTwo 5 (841 * … * 29)
    → addTwo 5 (24389 * … * 29)
    …
    → addTwo 5 (12693446555…)
    → 5 + 2
    → 7
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
```

Why is this useful?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
addTwo 5 (29^210)
    → 5 + 2
    → 7
```

Why is this useful?

```
addTwoSometimes :: Int -> Int -> Bool -> Int
addTwoSometimes x y b =
    if b then
        x + y
    else
        x + 2
```

Why is this useful?

```
addTwoSometimes :: Int -> Int -> Bool -> Int
addTwoSometimes x y b =
    if b then
        x + y
    else
        x + 2
```

```
addTwoSometimes 5 (29^210) complexBooleanFormula
```

Why is this useful?

```
(&&) :: Bool -> Bool -> Bool
True && x = x
False && _ = False
```

```
False && complexBooleanFormula
```

What’s the catch?

```
sum_debug :: List Int -> Int
sum_debug xs =
    print “Running sum!”
    sum xs


app_debug :: List a -> List a -> List a
app_debug xs ys =
    print “Running append!”
    xs ++ ys
```

Note: this is not valid Haskell

What's the catch?

```
sum_debug :: List Int -> Int
sum_debug xs =
    print "Running sum!"
    sum xs
```

```
> sum_debug (app_debug [6] [4])
```

What is the output in a strict language?

```
app_debug :: List a -> List a -> List a
app_debug xs ys =
    print "Running append!"
    xs ++ ys
```

Note: this is not valid Haskell

What’s the catch?

```
sum_debug :: List Int -> Int
sum_debug xs =
    print “Running sum!”
    sum xs
```

```
> sum_debug (app_debug [6] [4])
Running append!
Running sum!
10
```

```
app_debug :: List a -> List a -> List a
app_debug xs ys =
    print “Running append!”
    xs ++ ys
```

Note: this is not valid Haskell

What's the catch?

```
sum_debug :: List Int -> Int
sum_debug xs =
    print "Running sum!"
    sum xs
```

```
> sum_debug (app_debug [6] [4])
```

What is the output in a lazy language?

```
app_debug :: List a -> List a -> List a
app_debug xs ys =
    print "Running append!"
    xs ++ ys
```

Note: this is not valid Haskell

What’s the catch?

```
sum_debug :: List Int -> Int
sum_debug xs =
    print “Running sum!”
    sum xs
```

```
> sum_debug (app_debug [6] [4])
Running sum!
Running append!
10
```

```
app_debug :: List a -> List a -> List a
app_debug xs ys =
    print “Running append!”
    xs ++ ys
```

Note: this is not valid Haskell

What's the catch?

Laziness makes it hard to reason about *side effects*. This is why it is pure.

Pure programs simply reduce terms to their simplest forms, without side effects

```
map (+1) [4,1,3] → 1+4 : map (+1) [1,3] → … → [5,2,4]
```

Side effect: any way the program interacts with the outside world

- Printing
- Reading from memory
- Crashing

What’s the catch?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
exp_debug :: Int -> Int -> Int
exp_debug a b =
    print “Running exp!”
    a^b
```

```
> addTwo 5 (exp_debug 29 210)
```

What will the output be?

What’s the catch?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
> addTwo 5 (exp_debug 29 210)

7
```

```
exp_debug :: Int -> Int -> Int
exp_debug a b =
    print “Running exp!”
    a^b
```

What’s the catch?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
exp_debug :: Int -> Int -> Int
exp_debug a b =
    error “crashing”
    a^b
```

```
> addTwo 5 (exp_debug 29 210)
```

What will the output be?

What’s the catch?

```
addTwo :: Int -> Int -> Int
addTwo x y = x + 2
```

```
> addTwo 5 (exp_debug 29 210)

7
```

```
exp_debug :: Int -> Int -> Int
exp_debug a b =
    error “crashing”
    a^b
```

What’s the catch?

Laziness makes it hard to reason about *side effects*.

Side effect: any way the program interacts with the outside world

- Printing
- Reading from memory
- Crashing

Haskell is (mostly) a *pure* language — it has (mostly) no side effects!

What’s the catch?

Laziness can increase space usage.

What's the catch?

```
foldl f z []     = z
foldl f z (x:xs) = let z' = z `f` x  in foldl f z' xs

foldl add 0 [1,2,3]
let x1 = 0+1 in (foldl add x1 [2,3])
let x1 = 0+1 in (let x2 = x1+2 in (foldl add x2 [3]))
let x1 = 0+1 in (let x2 = x1+2 in (let x3 = x2+3 in (foldl add x3 [])))
let x1 = 0+1 in (let x2 = x1+2 in (let x3 = x2+3 in x3))
let x1 = 0+1 in (let x2 = x1+2 in (x2+3))
let x1 = 0+1 in ((x1+2)+3)
((0+1)+2)+3
(1+2)+3
3+3
6
```

Fold

`foldl'` removes some of the laziness

```
foldl' add 0 [1,2,3]
foldl' add (0 + 1) [2,3]
foldl' add 1 [2,3]
foldl' add (1 + 2) [3]
foldl' add 3 [3]
foldl' add (3 + 3) []
foldl' add 6 []
6
```

Strict Evaluation: !

- We can use the “bang” operator `!` on arguments (and sometimes operators) to force Haskell to evaluate them strictly

```
addTwo :: Int -> Int -> Int
addTwo x !y = x + 2


addTwo 5 (29^210)
= addTwo 5 (29 * 29 * … * 29)
= addTwo 5 12693446555…
= 5 + 2
= 7
```

Terminology

- Call-by-value ~ strict
- Call-by-name ~ lazy
  - Call-by-need ~ lazy with caching (what Haskell does!)

```
triple :: Int -> Int
triple x = x + x + x


triple (29^210)
= 29^210 + 29^210 + 29^210
= 12693446555… + 12693446555… + 12693446555…
```

Infinite Data Structures

Laziness lets us operate on infinite data structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
1 : 2 : map (1+) (map (1+) posInts)
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
1 : 2 : map (1+) (map (1+) posInts)
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
1 : 2 : map (1+) (map (1+) posInts)
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
1 : 2 : map (1+) (map (1+) posInts)
1 : 2 : map (1+) (map (1+) (1 : map (1+) posInts))
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
1 : 2 : map (1+) (map (1+) posInts)
1 : 2 : map (1+) (map (1+) (1 : map (1+) posInts))
1 : 2 : map (1+) (2 : (map (1+) (map (1+) posInts)))
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
1 : 2 : map (1+) (map (1+) posInts)
1 : 2 : map (1+) (map (1+) (1 : map (1+) posInts))
1 : 2 : map (1+) (2 : (map (1+) (map (1+) posInts)))
1 : 2 : 3 : map (1+) (map (1+) (map (1+) posInts))
```

Infinite Data Structures

```
posInts :: [Int]
posInts = 1 : map (1 +) posInts
```

```
posInts
1 : map (1+) posInts
1 : map (1+) (1 : map (1+) posInts)
1 : 2 : map (1+) (map (1+) posInts)
1 : 2 : map (1+) (map (1+) (1 : map (1+) posInts))
1 : 2 : map (1+) (2 : (map (1+) (map (1+) posInts)))
1 : 2 : 3 : map (1+) (map (1+) (map (1+) posInts))
```

The amount of a term that we need to evaluate is called the *demand*.

Are infinite data structures used in practice?

Yes!

- Generators – “almost-sure termination”
  - there exists a nonterminating execution path, but it happens with probability 0
- Games – can have state space so large it might as well be infinite
- Elegant representations of some lists
- Streams

Streams

Streams are inherently infinite, but laziness lets us manipulate them.

```
data Stream a = Cons a (Stream a)
```

Streams

Exercise:

Implement `take`, but for streams. (It should output a list.)

Streams

Exercise:

Implement `repeat`, but for streams.

Streams

Exercise:

Implement a function that turns a list into a stream.
If the list is finite, all elements after should be the input default element.

Hint: use foldr and streamRepeat.